COMP2040 – Operating Systems

Homework 01: Process Synchronization and Deadlocks

Overview

This homework covers Lectures 5 and 6 and is worth **100 points**, plus an optional **10-point bonus**. Use the notation from the lectures and show your working; a final answer without steps gets little credit. You may discuss ideas with classmates, but write your own answers.

Submission

Upload **one file** to Canvas before the deadline shown there: `StudentID_HW01.zip`, containing your answers `StudentID_HW01.pdf` and, if you did the bonus, your `hw1_buffer.c`, for example with `zip StudentID_HW01.zip StudentID_HW01.pdf hw1_buffer.c`. Scanned handwritten tables and graphs are fine if they are readable.

Part A: Synchronization (50 points)

Question 1: Race condition (10 points)

One seat is left (`seats = 1`) and two threads A and B run the code below to book it. `r` is a local variable, so each thread has its own copy.

```
if (seats > 0) {        /* check  */
    r = seats;          /* read   */
    r = r - 1;          /* update */
    seats = r;          /* write  */
    printf("booked\n");
}
```

(a) Give an interleaving of A and B in which *both* print `booked`. Which final values of `seats` are possible when this happens? (5)

(b) Where must `lock(m)` and `unlock(m)` go so that at most one booking succeeds? Why is locking only the read, update and write lines not enough? (5)

Question 2: Critical-section requirements (15 points)

Two threads T0 and T1 use each protocol below to protect a critical section. In each, `i` is the thread's own number and `j = 1 - i`. Initially `turn = 0`, `busy = false` and `flag[0] = flag[1] = false`.

```
/* Protocol 1 */  while (turn != i) { }
                  critical_section();  turn = j;
/* Protocol 2 */  while (busy) { }  busy = true;
                  critical_section();  busy = false;
/* Protocol 3 */  flag[i] = true;  while (flag[j]) { }
                  critical_section();  flag[i] = false;
```

(a) For protocol 1, name the requirement (mutual exclusion, progress or bounded waiting) that can fail, and give a short execution that shows it. (4)

(b) The same for protocol 2. (4)

(c) The same for protocol 3. Does protocol 3 guarantee mutual exclusion? (4)

(d) Peterson's solution adds `turn = j;` after `flag[i] = true;` and waits `while (flag[j] && turn == j)`. In one or two sentences, explain why the failure you found in protocol 3 cannot happen. (3)

Question 3: Semaphores and the bounded buffer (15 points)

A buffer with $N = 3$ slots uses `empty = 3`, `full = 0` and `mutex = 1` with the producer and consumer code of Lecture 5 (slide 52). P is the producer, C1 and C2 are consumers, and P produces items 1, 2, 3, 4 and 5, in that order. As on slide 34, a signal to a semaphore with a waiting thread hands the permit to that thread, so the value does not change. Give the state after each step, including the work of any thread the step wakes (buffer: oldest item first).

Step	Event	empty	full	Buffer	Waiting
0	Initially	3	0	[ ]	–
1	C1 calls remove(), buffer empty	3	0	[ ]	C1 on `full`
2	P inserts item 1; C1 wakes	_____	_____	_____	_____
3	C1 finishes removing item 1	_____	_____	_____	_____
4	P inserts item 2	_____	_____	_____	_____
5	P inserts item 3	_____	_____	_____	_____
6	P inserts item 4	_____	_____	_____	_____
7	P calls insert(item 5)	_____	_____	_____	_____
8	C2 removes one item; P wakes	_____	_____	_____	_____

(a) Complete the table. (9)

(b) At which steps does a thread block, on which semaphore, and which later step wakes it? (3)

(c) Suppose a consumer called `wait(mutex)` *before* `wait(full)`. What would happen at step 1 and afterwards? (3)

Question 4: Condition variables (10 points)

A monitor-style buffer uses a mutex `m` and a condition variable `not_empty`; `put()` inserts an item, increments `count` and signals `not_empty` while holding `m`. This `get()` has a bug:

```
int get(void) {
    lock(m);
    if (count == 0)
        wait(not_empty, m);      /* releases m; reacquires it on return */
    int item = remove_item();    /* count = count - 1 */
    unlock(m);
    return item;
}
```

(a) The buffer is empty. Give an execution with one producer and two consumers in which a consumer removes from the empty buffer. (6)

(b) Correct the code, and explain why your version is safe even if a thread wakes up without any signal (a spurious wake-up). (4)

Part B: Deadlocks (50 points)

Question 5: Banker's algorithm (25 points)

A system has three resource types with $(A, B, C) = (11, 13, 5)$ instances.

	Allocation			Max			Available		
Process	A	B	C	A	B	C	A	B	C
P0	3	2	3	8	3	4	2	4	1
P1	0	3	1	3	4	3			
P2	1	1	0	4	5	2			
P3	3	3	0	6	7	1			
P4	2	0	0	3	0	0			

(a) Compute the Need matrix. (5)
(b) Run the safety algorithm. For each step, give the process chosen, its Need, and Work before and after it releases its allocation. Is the state safe? Give a safe sequence. (8)
(c) P3 requests $(0, 1, 1)$. Apply the resource-request algorithm: check both bounds, write the tentative Available, Allocation and Need, and decide whether the request is granted. (6)
(d) Starting again from the *original* state, P0 requests the same vector $(0, 1, 1)$. Show that the request passes both bounds but must wait. Why is the same request safe for P3 and unsafe for P0? (6)

Question 6: Deadlock detection and recovery (15 points)

Three resource types have $(A, B, C) = (2, 6, 7)$ instances. The snapshot below shows the *current* allocations and outstanding requests.

	Allocation			Request			Available		
Process	A	B	C	A	B	C	A	B	C
P0	0	1	2	0	0	0	0	1	0
P1	0	1	1	0	2	1			
P2	0	0	2	2	1	1			
P3	1	1	2	1	0	2			
P4	1	2	0	2	2	0			

(a) Run the multi-instance detection algorithm, showing Work at each step. Is the system deadlocked? If so, which processes are in the deadlocked set? (7)
(b) The operating system will abort exactly one deadlocked process and release its resources. For each candidate, say whether aborting it alone ends the deadlock, and justify your answer. Which one do you choose? (8)

Question 7: Resource-allocation graphs (10 points)

(a) Resource R1 has one instance and R2 has two. T1 holds one instance of R2 and requests R1; T2 holds R1 and requests R2; T3 holds the other instance of R2 and requests nothing. Draw the resource-allocation graph. Does it contain a cycle? Is the system deadlocked? Then T3 also requests R1: answer both questions again. (5)
(b) Give a deadlock example of your own, not one from the lectures (road junction, two locks, dining philosophers). Show that the four necessary conditions hold, and explain how you would break one of them. (5)

Bonus: a bounded buffer as a monitor (10 points)

Complete `put()` and `get()` in `hw1_buffer.c` with the mutex `m` and the condition variables `not_full` and `not_empty`, as in Lecture 5 (slide 56). The build command is at the top of the file, and the test in `main()` must print `OK`. Put the output in your PDF and explain briefly why both waits are inside a `while` loop.